

Plan de Entrega Grupal — Proyecto Integrador Backend

API REST: FastAPI · SQLAlchemy · PostgreSQL

Materia: Programación IV — Tecnicatura Universitaria en Programación

Equipo: Federico Frankenberger · Emilia Barros · Miguel Barrera · Guadalupe Maricchiolo

Fecha de elaboración: Abril 2026

1. Estrategia de Control de Versiones (Git / GitHub)

1.1. Criterio de División del Trabajo

El código se divide en cuatro partes que siguen el orden natural de dependencias de la arquitectura: primero la infraestructura (setup y base de datos), luego los modelos (entidades de la BD), después la lógica de negocio (schemas + services) y por último los controladores HTTP (routers + punto de entrada). Cada integrante sube su parte solo cuando la anterior ya fue pusheada, garantizando un historial coherente.

1.2. Instrucciones Previas — Todos los integrantes

Antes del primer commit, asegurarse de que el repositorio tenga un archivo .gitignore que excluya .env, .venv/ y __pycache__/. Clonar el repositorio y posicionarse en la raíz del mismo (Primer_Parcial_Prog4/). Todos los comandos de git se ejecutan desde esa carpeta raíz; los paths de archivos comienzan con backend/.

Importante: El archivo backend/.env nunca debe commitearse. Crear un archivo backend/.env.example con las variables sin valores reales (ej: DATABASE_URL=) y commitearlo en su lugar.

1.3. Commit 1 — Federico Frankenger (Setup + Base de Datos + UoW)

Archivos a commitear:

- backend/requirements.txt
- backend/.env.example
- backend/app/__init__.py
- backend/app/database.py
- backend/app/uow/__init__.py
- backend/app/uow/uow.py

Configurar identidad git (una sola vez):

```
git config user.name "Federico Frankenger"
git config user.email "federico@email.com"
```

Agregar archivos al staging:

```
git add backend/requirements.txt backend/.env.example
git add backend/app/__init__.py backend/app/database.py
git add backend/app/uow/
```

Crear el commit:

```
git commit -m "feat: configurar entorno de desarrollo, conexion a PostgreSQL y patron Unit of Work"
```

Subir al repositorio remoto:

```
git push origin main
```

1.4. Commit 2 — Emilia Barros (Modelos SQLAlchemy)

Archivos a commitear (luego de hacer git pull):

- backend/app/models/__init__.py
- backend/app/models/categoria.py
- backend/app/models/ingrediente.py
- backend/app/models/producto.py
- backend/app/models/producto_categoria.py
- backend/app/models/producto_ingrediente.py

Actualizar la rama local:

```
git pull origin main
```

Configurar identidad git:

```
git config user.name "Emilia Barros"
git config user.email "emilia@email.com"
```

Agregar archivos:

```
git add backend/app/models/
```

Crear el commit:

```
git commit -m "feat: implementar modelos SQLAlchemy con relaciones 1:N y N:N para Categoria,
Ingrediente, Producto y tablas intermedias"
```

Subir:

```
git push origin main
```

1.5. Commit 3 — Miguel Barrera (Schemas + Services)

Archivos a commitear (luego de hacer git pull):

- backend/app/schemas/__init__.py
- backend/app/schemas/categoria.py
- backend/app/schemas/ingrediente.py
- backend/app/schemas/producto.py
- backend/app/services/__init__.py
- backend/app/services/categoria_service.py
- backend/app/services/ingrediente_service.py
- backend/app/services/producto_service.py

Actualizar la rama local:

```
git pull origin main
```

Configurar identidad git:

```
git config user.name "Miguel Barrera"
git config user.email "miguel@email.com"
```

Agregar archivos:

```
git add backend/app/schemas/ backend/app/services/
```

Crear el commit:

```
git commit -m "feat: agregar schemas Pydantic y logica de negocio de los servicios para los
modulos Categoria, Ingrediente y Producto"
```

Subir:

```
git push origin main
```

1.6. Commit 4 — Guadalupe Maricchiolo (Routers + main.py)

Archivos a commitear (luego de hacer git pull):

- backend/app/routers/__init__.py
- backend/app/routers/categorias.py
- backend/app/routers/ingredientes.py
- backend/app/routers/productos.py
- backend/app/main.py

Actualizar la rama local:

```
git pull origin main
```

Configurar identidad git:

```
git config user.name "Guadalupe Maricchiolo"
git config user.email "guadalupe@email.com"
```

Agregar archivos:

```
git add backend/app/routers/ backend/app/main.py
```

Crear el commit:

```
git commit -m "feat: implementar routers con endpoints CRUD, validaciones con Annotated/Path/Query
e integracion final en main.py"
```

Subir:

```
git push origin main
```

1.7. Historial Esperado en GitHub

Una vez completados los cuatro pasos, el historial de commits en GitHub debe verse de la siguiente manera (del más reciente al más antiguo):

Autor	Mensaje de commit
Guadalupe Maricchiolo	feat: implementar routers con endpoints CRUD, validaciones con Annotated/Path/Query e integracion final
Miguel Barrera	feat: agregar schemas Pydantic y logica de negocio de los servicios para los modulos Categoria, Ingrediente
Emilia Barros	feat: implementar modelos SQLAlchemy con relaciones 1:N y N:N para Categoria, Ingrediente, Producto y t
Federico Frankenberger	feat: configurar entorno de desarrollo, conexion a PostgreSQL y patron Unit of Work

2. Guion para el Video de Presentación

2.1. Información General del Video

Duración total máxima	10 minutos
Formato sugerido	Grabación de pantalla con voz en off (OBS, Loom o Clipchamp)
Resolución recomendada	1920x1080 (Full HD)
Herramienta para mostrar la BD	pgAdmin 4 o DBeaver
Herramienta para probar endpoints	Swagger UI en http://127.0.0.1:8000/docs
Orden de intervenciones	Guadalupe (intro) → Federico → Emilia → Miguel → Guadalupe (demo + cierre)

2.2. Distribución de Tiempos

Segmento	Responsable	Duración	Tiempo en video
Introducción general	Guadalupe Maricchiolo	35 seg	0:00 – 0:35
Setup, BD y UoW	Federico Frankenger	2 min	0:35 – 2:35
Modelos SQLModel	Emilia Barros	2 min 20 seg	2:35 – 4:55
Schemas y Services	Miguel Barrera	2 min 25 seg	4:55 – 7:20
Routers y Demo Swagger	Guadalupe Maricchiolo	2 min 05 seg	7:20 – 9:25
Cierre grupal	Todos (Guadalupe lidera)	35 seg	9:25 – 10:00

2.3. Guion Detallado

INTRODUCCIÓN — Guadalupe Maricchiolo

Tiempo: 0:00 – 0:35 | 35 segundos

Lo que debe mostrarse en pantalla:

- Mostrar el repositorio en GitHub con el historial de commits visible.
- Luego cambiar a la estructura de carpetas del proyecto en VS Code.

Texto a decir:

Buenos días, buenos tardes. Somos el equipo conformado por Federico Frankenger, Emilia Barros, Miguel Barrera y yo, Guadalupe Maricchiolo. En este video presentamos nuestro proyecto integrador de Programación IV: una API REST para la gestión de un catálogo de productos, desarrollada con FastAPI, SQLModel y PostgreSQL. La arquitectura está organizada en cinco capas: modelos, schemas, servicios, routers y una unidad de trabajo para la gestión de sesiones. Cada integrante explica la capa que desarrolló. Empezamos.

SEGMENTO 1 — Federico Frankenger (Setup, Base de Datos y UoW)

Tiempo: 0:35 – 2:35 | 2 minutos

Lo que debe mostrarse en pantalla:

- [0:35] Mostrar backend/requirements.txt en el editor.
- [0:55] Mostrar backend/app/database.py completo.
- [1:20] Mostrar backend/app/uow/uow.py completo.
- [1:50] Abrir pgAdmin o DBeaver y mostrar la base de datos 'parcial_db' conectada (puede estar vacía aún; alcanza con mostrar la conexión activa).
- [2:20] Mostrar la terminal con el servidor levantado: 'fastapi dev app/main.py'.

Texto a decir:

Hola, soy Federico. Me encargué del setup inicial del proyecto y de la infraestructura de base de datos.

En requirements.txt declaramos las cuatro dependencias del proyecto: fastapi con su variante 'standard' que incluye el servidor uvicorn, sqlmodel que nos da el ORM y la validación en una sola librería, psycopg2-binary que es el adaptador para conectarnos a PostgreSQL, y python-dotenv para leer las variables de entorno desde el archivo .env sin hardcodear la cadena de conexión en el código.

En database.py configuramos el motor de base de datos usando create_engine de SQLAlchemy. La URL se lee de la variable de entorno DATABASE_URL. La función crear_tablas llama a SQLAlchemy.metadata.create_all, que crea físicamente todas las tablas definidas por los modelos al iniciar el servidor.

Por último, en uow.py implementé el patrón Unit of Work. Es un context manager: al entrar al bloque 'with' se abre una sesión de base de datos, y al salir se hace commit automático si todo salió bien, o rollback si ocurrió algún error. Esto garantiza que ninguna operación quede a medias en la base de datos.

SEGMENTO 2 — Emilia Barros (Modelos SQLAlchemy)

Tiempo: 2:35 – 4:55 | 2 minutos 20 segundos

Lo que debe mostrarse en pantalla:

- [2:35] Mostrar backend/app/models/categoria.py.
- [3:00] Mostrar backend/app/models/ingrediente.py.
- [3:15] Mostrar backend/app/models/producto.py, enfocando las dos relaciones Relationship.
- [3:45] Mostrar backend/app/models/producto_categoria.py y producto_ingrediente.py.
- [4:20] Mostrar pgAdmin: ejecutar la app una vez y mostrar las 5 tablas creadas en la base de datos con sus columnas y claves foráneas.

Texto a decir:

Hola, soy Emilia. Yo desarrollé la capa de modelos, que representa la estructura de la base de datos.

Tenemos cinco modelos. Categoria define la tabla 'categoria' con los campos id, parent_id que es una FK auto-referencial para soportar jerarquías de categorías padre e hija, nombre con restricción UNIQUE, descripcion e imagen_url.

Ingrediente es similar: id, nombre único y descripcion. Es una entidad global, no se duplica por producto.

Producto es la entidad central. Tiene precio_base, stock_cantidad y disponible. Lo más importante son las dos relaciones que declaré con Relationship: categorias_link e ingredientes_link. Cada una apunta a una tabla intermedia, que es como SQLAlchemy implementa las relaciones N:N. Le agregué cascade='all, delete-orphan' para que al borrar un producto se borren automáticamente sus vínculos.

ProductoCategoria y ProductoIngrediente son esas tablas intermedias. Tienen clave primaria compuesta por producto_id más categoria_id, o producto_id más ingrediente_id respectivamente. ProductoIngrediente agrega el campo es_removible, que indica si el cliente puede pedir que se quite ese ingrediente de su pedido.

Para evitar importaciones circulares entre los modelos, usamos 'from __future__ import annotations' con bloques TYPE_CHECKING. Pueden ver las cinco tablas ya creadas en pgAdmin.

SEGMENTO 3 — Miguel Barrera (Schemas y Services)

Tiempo: 4:55 – 7:20 | 2 minutos 25 segundos

Lo que debe mostrarse en pantalla:

- [4:55] Mostrar backend/app/schemas/categoria.py, señalando CategoriaCreate y CategoriaRead.
- [5:25] Mostrar backend/app/schemas/producto.py, enfocando ProductoCreate con categoria_ids e ingredientes, y ProductoRead con las listas anidadas.
- [5:55] Mostrar backend/app/services/categoria_service.py, explicando obtener_categorias con el filtro por nombre.
- [6:25] Mostrar backend/app/services/producto_service.py, enfocando crear_producto (el flush() y los vínculos) y _construir_producto_read.
- [7:00] Mostrar eliminar_categoria con el bloque if productos_link: raise ValueError.

Texto a decir:

Hola, soy Miguel. Me encargué de los schemas de validación y de los servicios donde vive la lógica de negocio.

Los schemas separan lo que entra de lo que sale. Por ejemplo, CategoriaCreate define qué datos necesita el cliente para crear una categoría, con sus validaciones: el nombre debe tener entre 2 y 100 caracteres. CategoriaRead define qué devolvemos nosotros: incluye el id pero nunca expone campos internos. Esto lo controla FastAPI con el atributo response_model en los routers.

Para Producto, el schema de entrada ProductoCreate incluye dos listas adicionales: categoria_ids, que son los IDs de las categorías a asignar, e ingredientes, que es una lista de objetos con ingrediente_id y el flag es_removible. El ProductoRead devuelve las categorías e ingredientes como objetos anidados.

En los servicios, cada función usa el Unit of Work. Por ejemplo, crear_producto hace un flush para obtener el ID del producto antes de crear los vínculos, y luego llama a _construir_producto_read, que accede a las relaciones mientras la sesión está abierta para poder hacer el lazy loading.

También manejamos un caso de integridad: si se intenta borrar una categoría que tiene productos, verificamos si tiene vínculos activos y lanzamos un ValueError, que el router convierte en una respuesta HTTP 409 Conflict.

SEGMENTO 4 — Guadalupe Maricchiolo (Routers y Demo en Swagger)

Tiempo: 7:20 – 9:25 | 2 minutos 5 segundos

Lo que debe mostrarse en pantalla:

- [7:20] Mostrar backend/app/routers/categorias.py, señalando el uso de Annotated, Query y Path en listar_categorias.
- [7:45] Mostrar backend/app/routers/productos.py, señalando los sub-endpoints POST /{id}/categorias y DELETE /{id}/ingredientes/{ing_id}.
- [8:05] Mostrar backend/app/main.py mostrando el lifespan y los include_router.
- [8:20] Abrir el navegador en http://127.0.0.1:8000/docs (Swagger UI).
- [8:25] Ejecutar POST /categorias con nombre='Hamburguesas' y mostrar respuesta 201.
- [8:35] Ejecutar POST /ingredientes con nombre='Queso cheddar'.
- [8:45] Ejecutar POST /productos con categoria_ids y ingredientes, mostrar respuesta 201 con los objetos anidados.
- [9:00] Ejecutar GET /productos?nombre=ham para demostrar el filtro.
- [9:10] Ejecutar DELETE /categorias/{id} con un id que tenga productos -> mostrar 409.
- [9:18] Ejecutar DELETE /productos/{id} -> mostrar 204 y confirmar en pgAdmin que las tablas intermedias se vaciaron.

Texto a decir:

Gracias Miguel. Yo, Guadalupe, implementé los routers que son los controladores HTTP, y también el main.py que

integra todo.

En los routers, una decisión clave fue usar `Annotated` junto con `Query` y `Path` de FastAPI para declarar las validaciones de cada parámetro directamente en la firma del endpoint. Por ejemplo, en `GET /categorias`, el parámetro `skip` tiene `ge=0` y `limit` tiene `ge=1` y `le=100`. FastAPI los valida antes de ejecutar cualquier código.

Para las relaciones N:N, agregué sub-endpoints: `POST /productos/{id}/categorias` para asignar una categoría a un producto, y `DELETE /productos/{id}/categorias/{cat_id}` para quitarla. Lo mismo para ingredientes.

En `main.py` el `lifespan` llama a `crear_tablas` al iniciar, y los tres `include_router` registran los prefijos `/categorias`, `/ingredientes` y `/productos`.

Pasemos a la demo. Abro Swagger en el puerto 8000. Primero creo una categoría 'Hamburguesas', devuelve 201 con el id generado. Luego creo el ingrediente 'Queso cheddar'. Ahora creo un producto pasando el `categoria_id` y el `ingrediente_id`: la respuesta 201 incluye los objetos anidados de categorías e ingredientes, que es justamente lo que va a consumir el frontend. Si filtro con `?nombre=ham`, el endpoint devuelve solo los productos que coinciden. Si intento borrar la categoría que tiene ese producto, recibo un 409 Conflict. Y si borro el producto primero, el cascade elimina automáticamente los vínculos.

CIERRE — Guadalupe Maricchiolo (en nombre del equipo)

Tiempo: 9:25 – 10:00 | 35 segundos

Lo que debe mostrarse en pantalla:

- Mostrar el historial de commits en GitHub con los cuatro autores visibles.
- Opcionalmente, mostrar el README.md del repositorio.

Texto a decir:

Con esto concluimos la presentación de nuestro proyecto. Implementamos una API REST funcional con persistencia real en PostgreSQL, relaciones 1:N y N:N con `SQLModel`, validaciones con `Pydantic`, separación de responsabilidades en capas y una gestión de transacciones mediante el patrón `Unit of Work`. Gracias por su atención.